

1) Design a machine learning model for spam email classification using labeled data.

(a) Explain the choice of algorithm (eg. Naive Bayes, SVM, or Logistic Regression)

(b) Describe the preprocessing steps such as tokenization, stop word removal, and vectorization.

(c) Discuss evaluation metrics such as accuracy, precision, recall, and F1-Score.

a) choice of algorithm.

* Naive Bayes is commonly used for spam email classification because:-

* It works well with text data.

* It is fast and computationally efficient.

* It assumes independence between words, making classification

* It perform well even with large datasets.

* SVM or Logistic Regression can also be used, but Naive Bayes is preferred for text classification.

b) Preprocessing steps.

1. Tokenization - split email text into individual words (tokens)

2. Lowercasing - convert all text to lowercase.

3. Stop word Removal - Remove common words like, the, is, and

4. Stemming / Lemmatization - Remove common words like, the, is, and convert words to their root form (eg, running $\rightarrow$ run)

5. Vectorization - convert text into numerical form using Bag of words (BOW) or TF-IDF.

C) Evaluation Metrics:

Accuracy :- Percentage of correctly classified emails.

Precision :- Percentage of predicted spam emails that are actually spam.

Recall :- Percentage of actual spam emails correctly identified.

F1-score :- Harmonic mean of precision and recall

Conclusion :-

- * In design of machine learning model for spam email classification, can accurately distinguish spam and non-spam emails by training on labeled data.
- * Using Naive Bayes along with preprocessing techniques such as tokenization, stop-word removal, and TF-IDF / Vectorization improve classification performance.

2) In the context of the candidate - elimination, in algorithm, consider a dataset with attributes: sky, Air Temp, Humidity, wind, water, forecast and target concept enjoy sport.

- Explain the Initialization of S (Specific boundary) and G (General boundary)
- Show how S and G change with each training example
- Compare candidate - Elimination with the find - S algorithm in terms of generalization capability.

a) Initialization of S and G

Specific Boundary (S):

starts with most specific hypothesis

Example: $S = \{ \langle \phi, \phi, \phi, \phi, \phi, \phi \rangle \}$

General boundary (G):

starts with most general hypothesis

Example: $G = \{ \langle ?, ?, ?, ?, ? \rangle \}$

b) changes in S and G with training examples:

Suppose the training examples are:

Example	sky	Air Temp	Humidity	wind	water	forecast	enjoy sport
1	Sunny	Warm	Normal	strong	Warm	Same	Yes
2	Sunny	Warm	High	strong	Warm	Same	Yes
3	Rainy	Cold	High	strong	Warm	change	No

After example (1) (positive)

$S = \langle \text{sunny, warm, normal, strong, warm, same} \rangle$

$G = \langle ?, ?, ?, ?, ?, ? \rangle$

After example 2 (positive):

$S = \langle \text{sunny, warm, ?, strong, warm, same} \rangle$

$G = \langle ?, ?, ?, ?, ?, ? \rangle$

After example 3 (Negative):

$S = \langle \text{sunny, warm, ?, strong, warm, same} \rangle$

$G = \{ \langle \text{sunny, ?, ?, ?, ?, ?} \rangle, \langle \text{?, warm, ?, ?, ?, ?} \rangle, \langle \text{?, ?, ?, ?, ?, same} \rangle \}$

The algorithm continues updating S and G until the version space is obtained.

g) candidate - Elimination vs find-S

Candidate Elimination	find-S
Maintains both S and G boundaries	Maintains only the most specific hypothesis
Learns from positive and Negative examples	Use only positive examples
produces all consistent hypothesis (version space)	produces a single hypothesis
Better generalization capability	Limited generalization
More computationally expensive	simple and fast